

Architekturdokumentation

Modul 3 – Objektorientierte Programmierung

Vererbung & Polymorphismus in Python

Verfasser: **Abdel El Amri**

Kurs: Angewandte Softwareentwicklung

Sommersemester 2026

Inhaltsverzeichnis

1	Allgemeine Erklärung von Vererbung und Polymorphismus	2
1.1	Vererbung	2
1.2	Polymorphismus	3
2	Darstellung und Erklärung der Anwendung	4
2.1	Aufbau und Funktionsweise	4
2.2	UML-Klassendiagramm	4
2.3	Ablaufdiagramm	6
3	Umsetzung in Python & Kritische Würdigung	8
3.1	Konkrete Umsetzung in der Anwendung	8
3.1.1	Vererbung	8
3.1.2	Polymorphismus	8
3.2	Python-Besonderheiten	9
3.2.1	Dynamic Typing	9
3.2.2	Duck Typing	9
3.2.3	Method Resolution Order (MRO)	9
3.3	Kritische Würdigung: Vor- und Nachteile	10
3.3.1	Vorteile der gewählten Architektur	10
3.3.2	Nachteile und potenzielle Alternativen	10

Kapitel 1

Allgemeine Erklärung von Vererbung und Polymorphismus

1.1 Vererbung

Vererbung (*engl. Inheritance*) ist ein grundlegendes Konzept der objektorientierten Programmierung. Es beschreibt die Möglichkeit, eine neue Klasse – die *Kindklasse* oder *Unterklasse* – auf Basis einer bereits bestehenden Klasse, der *Elternklasse* oder *Basisklasse*, zu definieren. Die Kindklasse übernimmt dabei automatisch alle Attribute und Methoden der Elternklasse und kann diese entweder direkt nutzen, erweitern oder überschreiben.

Formal spricht man von einer **Is-a-Beziehung**: Eine Katze *ist ein* Tier; ein Sales-Mitarbeiter *ist ein* Mitarbeiter. Der wesentliche Vorteil besteht in der **Wiederverwendbarkeit** von Code: Gemeinsame Logik wird einmalig in der Elternklasse definiert und von allen Kindklassen geerbt, ohne dupliziert werden zu müssen.

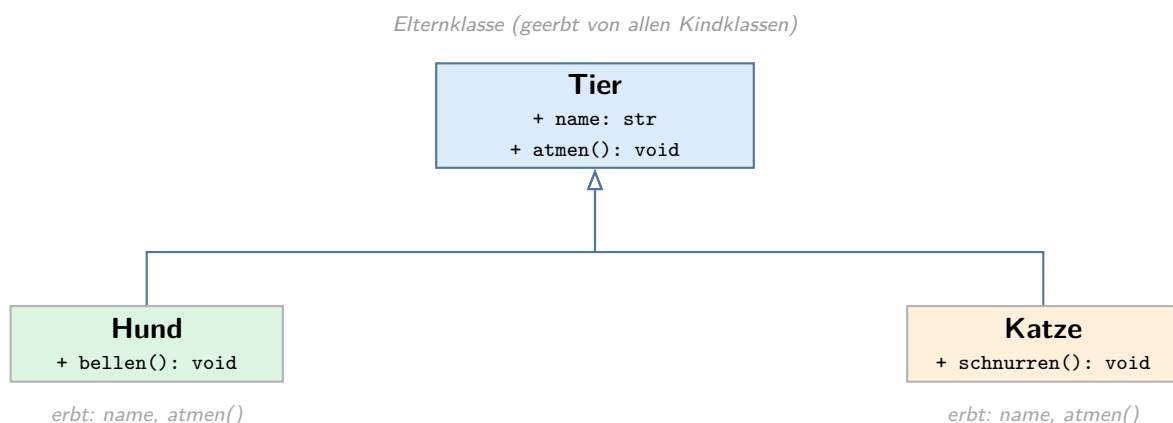


Abbildung 1.1: Vererbung: Hund und Katze erben gemeinsame Eigenschaften von Tier

1.2 Polymorphismus

Polymorphismus (*griech. Vielgestaltigkeit*) beschreibt die Fähigkeit, dass Objekte verschiedener Klassen auf *denselben* Methodenaufruf *unterschiedlich* reagieren. Das bedeutet: Der aufrufende Code muss nicht wissen, mit welchem konkreten Typ er es zu tun hat – er ruft schlicht eine Methode auf, und jedes Objekt antwortet gemäß seiner eigenen Implementierung.

Man unterscheidet zwei Hauptformen:

- **Überschreiben** (*Method Overriding*): Eine Kindklasse definiert eine Methode der Elternklasse neu. Die zur Laufzeit aufgerufene Version richtet sich nach dem tatsächlichen Typ des Objekts.
- **Überladen** (*Method Overloading*): Mehrere Methoden mit gleichem Namen, aber unterschiedlichen Parametern. In Python wird dies typischerweise über Standardargumente realisiert.

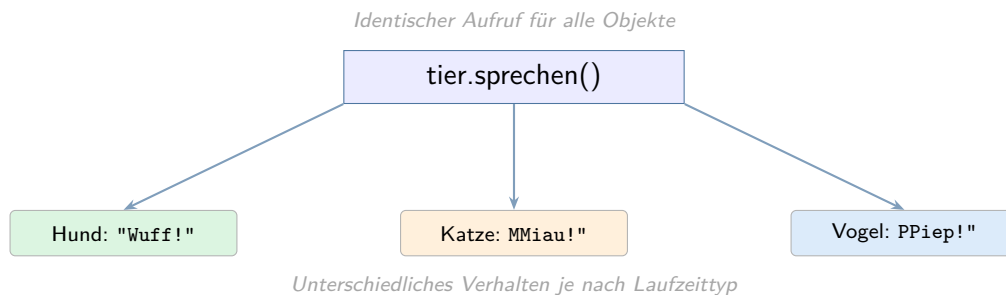


Abbildung 1.2: Polymorphismus: Gleicher Aufruf – verschiedene Reaktionen je nach Klasse

Kapitel 2

Darstellung und Erklärung der Anwendung

2.1 Aufbau und Funktionsweise

Die Anwendung modelliert die Personalstruktur des fiktiven Unternehmens *Mustermann GmbH*. Sie besteht aus fünf Python-Klassen, die strikt in separate Dateien aufgeteilt sind, sowie einer zentralen Steuerdatei (`main.py`) und einer REST-API (`api.py`).

Datei	Verantwortlichkeit
<code>unternehmen.py</code>	Basisklasse: Firmenname und Kalender
<code>mitarbeiter.py</code>	Abstrakte Mitarbeiter-Ebene; definiert polymorphe Schnittstelle
<code>sales.py</code>	Vertrieb: Wochenend-Berechtigung, Provisionsberechnung
<code>admin.py</code>	Administration: Bürozeiten-Beschränkung, Ticket-Erstellung
<code>hr.py</code>	Human Resources: satirischer Problem-Counter
<code>main.py</code>	Zentrale Steuerung: Instanziierung und Programmablauf
<code>api.py</code>	FastAPI-REST-Schnittstelle, Port 8080, In-Memory-Datenhaltung

Tabelle 2.1: Dateien und ihre Verantwortlichkeiten

2.2 UML-Klassendiagramm

Das folgende Diagramm zeigt alle fünf Klassen mit ihren vollständigen Attributen und Methoden. Offene Dreiecke an den Pfeilen kennzeichnen Vererbungsbeziehungen (*Is-a*).

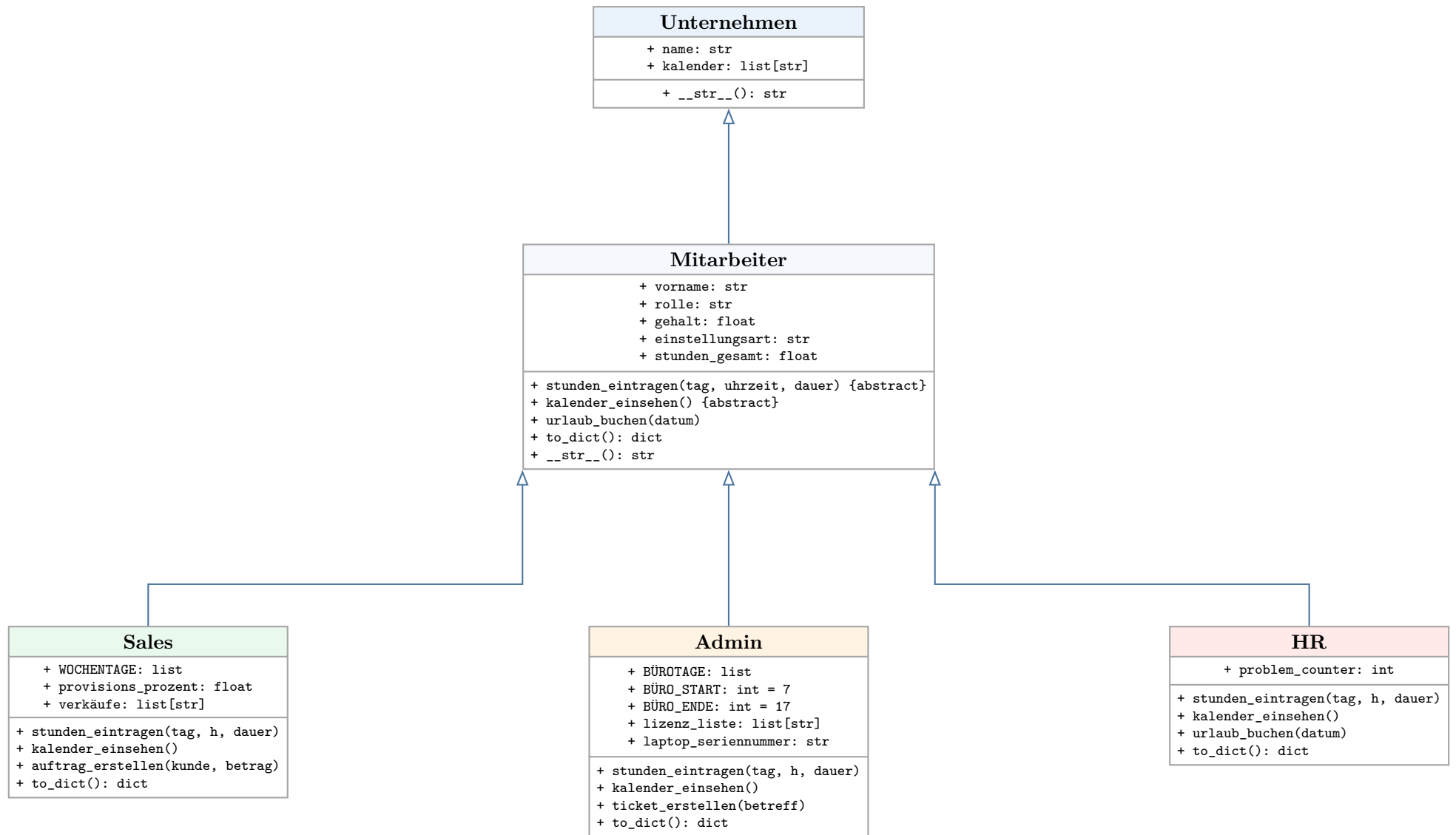


Abbildung 2.1: Vollständiges UML-Klassendiagramm der Anwendung

2.3 Ablaufdiagramm

Abbildung 2.2 zeigt als Flowchart, wie `main.py` die Klassen instanziiert und wie die polymorphen Methodenaufrufe zur Laufzeit abhängig vom Mitarbeitertyp unterschiedliche Pfade nehmen.

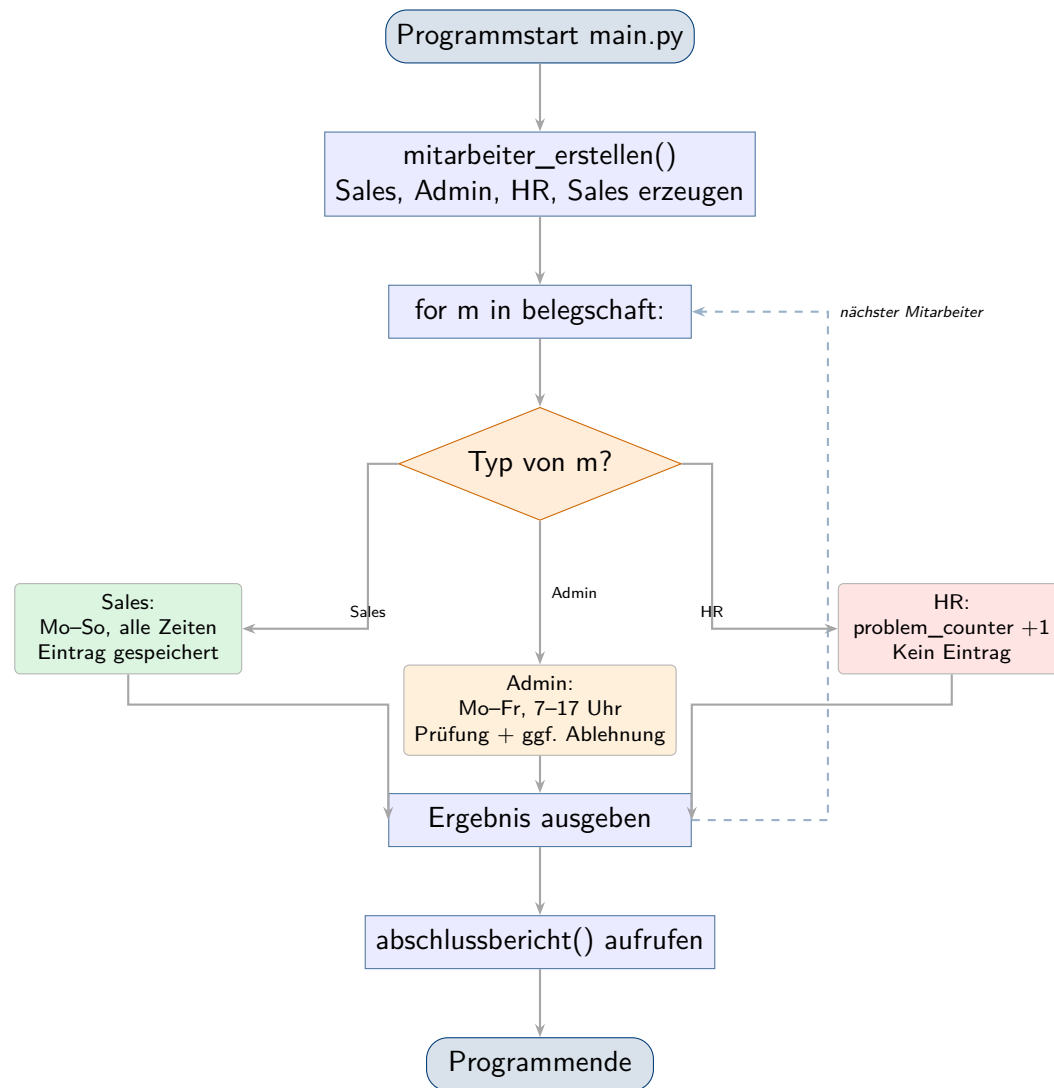


Abbildung 2.2: Ablaufdiagramm: Polymorphe Methodenverteilung in main.py

Kapitel 3

Umsetzung in Python & Kritische Würdigung

3.1 Konkrete Umsetzung in der Anwendung

3.1.1 Vererbung

Die Vererbungskette der Anwendung besteht aus drei Ebenen: Unternehmen → Mitarbeiter → {Sales, Admin, HR}.

`super().__init__()` sorgt dafür, dass die Initialisierungslogik jeder Elternklasse vollständig ausgeführt wird, bevor die Kindklasse eigene Attribute setzt. Dadurch entsteht eine lückenlose Attributkette ohne Duplikate:

```
1 class Sales(Mitarbeiter):
2     def __init__(self, vorname, firmenname, gehalt,
3                 einstellungsart, provisions_prozent=5.0):
4         # ruft Mitarbeiter.__init__ auf, der wiederum Unternehmen.
5         __init__ ruft
6         super().__init__(vorname, firmenname, "Sales",
7                           gehalt, einstellungsart)
8         self.provisions_prozent = provisions_prozent
9         self.verkaeufe: list[str] = []
```

Listing 3.1: Vererbungskette mit `super()` – sales.py

3.1.2 Polymorphismus

Mitarbeiter deklariert `stunden_eintragen()` und `kalender_einsehen()` als abstrakte Schnittstelle, indem es bei Aufruf einen `NotImplementedError` auslöst. Jede Kindklasse muss diese Methoden zwingend implementieren:

```
1 def stunden_eintragen(self, tag: str, uhrzeit: int, dauer: float =
2     1.0):
3     raise NotImplementedError(
4         f"{self.rolle} hat keine Implementierung fuer
5         stunden_eintragen."
6     )
```

Listing 3.2: Abstrakte Schnittstelle – mitarbeiter.py

Durch Polymorphismus bleibt `main.py` vollständig unabhängig vom konkreten Typ jedes Mitarbeiters:

```

1 for m in belegschaft:                # Typ von m ist zur Kompilierzeit
    unbekannt
2     m.stunden_eintragen("Montag", 9, 8.0)
3     # Sales    -> Eintrag gespeichert (Wochenende erlaubt)
4     # Admin    -> Pruefung: nur Mo-Fr, 7-17 Uhr
5     # HR       -> problem_counter += 1, kein Eintrag

```

Listing 3.3: Polymorpher Aufruf – main.py

3.2 Python-Besonderheiten

3.2.1 Dynamic Typing

Python ist **dynamisch typisiert**: Variablentypen werden erst zur Laufzeit bestimmt, nicht zur Kompilierzeit. Dies bedeutet, dass eine Variable in einer Iteration ein `Sales`-Objekt halten kann, in der nächsten ein `Admin`-Objekt – Python prüft den Typ beim tatsächlichen Aufruf. Type Hints (`list[Mitarbeiter]`) verbessern die Lesbarkeit, erzwingen aber keine Typbindung zur Laufzeit.

3.2.2 Duck Typing

Python folgt dem Prinzip des **Duck Typing**: „Wenn es wie eine Ente quakt und wie eine Ente läuft, ist es eine Ente.“ Das bedeutet, dass Python beim Aufruf von `m.stunden_eintragen()` nicht prüft, ob `m` eine Instanz von `Mitarbeiter` ist – es prüft einzig, ob das Objekt eine Methode dieses Namens besitzt. Polymorphismus funktioniert in Python deshalb auch ohne formale Vererbungsbeziehungen; Vererbung schafft hier zusätzlich eine semantisch saubere Klassenhierarchie.

3.2.3 Method Resolution Order (MRO)

Python löst Methodenaufrufe nach der **C3-Linearisierung** auf. Bei einem Aufruf `m.stunden_eintragen()` durchsucht Python die Klassenhierarchie des Objekts von der spezifischsten zur allgemeinsten Klasse. Die MRO lässt sich mit `Klasse.__mro__` inspizieren:

```

1 >>> Sales.__mro__
2 (<class 'Sales'>, <class 'Mitarbeiter'>, <class 'Unternehmen'>, <class
    'object'>)

```

Listing 3.4: MRO-Inspektion zur Laufzeit

Python findet `stunden_eintragen` zuerst in `Sales` und ruft diese Implementierung auf, ohne die Elternklassen weiter zu durchsuchen. `super()` hingegen überspringt die aktuelle Klasse explizit und startet die Suche bei der nächsten Klasse in der MRO.

3.3 Kritische Würdigung: Vor- und Nachteile

3.3.1 Vorteile der gewählten Architektur

1. **Keine Codeduplikation.** Gemeinsame Attribute (`vorname`, `gehalt`, `stunden_gesamt`) und Methoden (`urlaub_buchen`, `to_dict`) sind einmalig in `Mitarbeiter` definiert und werden von allen Rollen geerbt.
2. **Offenes Erweiterungsmodell.** Das Hinzufügen einer neuen Rolle (z. B. `Projektmanager`) erfordert ausschließlich eine neue Datei, die `Mitarbeiter` erbt und die Schnittstelle implementiert. `main.py`, `api.py` und alle anderen Dateien bleiben unverändert – das *Open/Closed Principle* wird eingehalten.
3. **Typsicherheit durch Schnittstelle.** Der `NotImplementedError` in `Mitarbeiter` erzwingt, dass jede neue Unterklasse die polymorphen Methoden implementiert. Vergessene Implementierungen werden sofort zur Laufzeit sichtbar.
4. **Lesbarkeit.** Der Code in `main.py` ist unabhängig von Rollenunterschieden. Eine schlanke `for`-Schleife ersetzt ansonsten notwendige `if/elif`-Kaskaden.

3.3.2 Nachteile und potenzielle Alternativen

1. **Tiefe Klassenhierarchie erhöht Komplexität.** Bei drei Vererbungsebenen müssen Entwickler die gesamte Hierarchie kennen, um das Verhalten einer Klasse zu verstehen. Bei weiterer Vertiefung kann dies zu dem bekannten Problem „*Fragile Base Class*“ führen: Änderungen in `Mitarbeiter` können unbeabsichtigt alle Unterklassen betreffen.
2. **Kein echtes Interface-Konzept in Python.** Python besitzt im Gegensatz zu Java oder C# keine formalen Interfaces. `NotImplementedError` ist ein Workaround; ein saubereres Mittel wäre die Nutzung des `abc`-Moduls (*Abstract Base Classes*):

```
1 from abc import ABC, abstractmethod
2
3 class Mitarbeiter(Unternehmen, ABC):
4     @abstractmethod
5     def stunden_eintragen(self, tag: str, uhrzeit: int, dauer:
6         float): ...
7
8     @abstractmethod
9     def kalender_einsehen(self): ...
```

Listing 3.5: Sauberere Alternative mit Abstract Base Classes

Mit `ABC` würde Python bereits bei der *Instanziierung* einer unvollständig implementierten Klasse eine `TypeError`-Ausnahme werfen – nicht erst beim ersten Methodenaufruf.

3. **Komposition als Alternative.** Für Szenarien, in denen sich Rollen und Berechtigungen häufig ändern oder kombinieren (z. B. ein Mitarbeiter mit Sales- und Admin-Rechten), wäre das *Kompositionsmuster* (*Composition over Inheritance*) flexibler: Ein

Mitarbeiter-Objekt würde ein austauschbares Rolle-Objekt halten, statt selbst eine Rolle zu sein. Für den klar abgegrenzten Anwendungsfall dieses Moduls ist Vererbung jedoch die passendere und leserlichere Lösung.

Verwendete Werkzeuge: Python 3.12 | FastAPI | Uvicorn | Pydantic (Entwicklung)
Claude Code (Implementierung) | Google Gemini (Anforderungsformulierung) | L^AT_EX
(Dokumentation)